Proyecto 2 — Reconocimiento con PDA y CYK

Universidad del Valle de Guatemala · Teoría de la Computación CC2019, sección 30 · Docente: Tomás Gálvez P. · Semestre 2, 2026

Integrante	Carné	Responsabilidad principal
David López	24730	PDA, extensión del proyecto 1, consola, repositorio
Carlos López	24531	CYK, integración, experimentos, comparación, informe, video

- Repositorio (privado): Repositorio del proyecto
- Video (YouTube, no listado, ≤ 10 min): Video de demostración

Informe final · Noviembre de 2026

Índice

1. Objetivo

2. La gramática G

3. PDA a partir de G

3.1 Extensión del autómata del proyecto 1

3.2 Definición formal

3.3 Simulación y evidencia

4. Algoritmo CYK

5. Ejecución

6. Tasas de crecimiento

6.1 CYK: desglose y cota ajustada

6.2 PDA: cota exponencial en general, polinomial en G

7. Experimentos reproducibles

7.1 Gráficas del conteo dominante

8. Discusión de desempeños

9. Validación

10. Referencias

1. Objetivo

Reconocer frases del lenguaje de la gramática G del enunciado con dos métodos independientes: un autómata de pila (PDA) obtenido de G, que extiende el autómata del proyecto 1, y el algoritmo CYK. Ambos registran los pasos que justifican su respuesta. Además se calcula la tasa de crecimiento de cada programa mediante conteo de operaciones y se comparan sus desempeños.

2. La gramática G

G = (V_G, Σ, S, R), con V_G = {S, VP, PP, NP, V, P, N, Det} y Σ = {cooks, drinks, eats, cuts, he, she, in, with, a, the, cat, dog, beer, cake, juice, meat, soup, fork, knife, oven, spoon}.

```
S  → NP VP              PP  → P NP              V   → cooks | drinks | eats | cuts
VP → VP PP | V NP       NP  → Det N | he | she  P   → in | with
VP → cooks | drinks | eats | cuts               Det → a | the
N  → cat | dog | beer | cake | juice | meat | soup | fork | knife | oven | spoon
```

8 variables, 21 terminales y 30 alternativas (5 binarias, 25 léxicas). G ya está en CNF. Está definida una sola vez en `recognizer/grammar.py` y la usan ambos reconocedores; las pruebas comparan esa definición con el enunciado. **Entrada:** cada palabra separada por espacios en blanco es un token y n es la cantidad de palabras. No se cambian mayúsculas ni puntuación: `He cooks` y `he cooks.` se rechazan. La cadena vacía se rechaza porque G no deriva ε.

3. PDA a partir de G

3.1 Extensión del autómata del proyecto 1

El modelo AFN del proyecto 1 se reutiliza con imports relativos y comentarios de procedencia en `recognizer/proyecto1/` (commit `750243e` de `primer-proyecto-tdc`). `AutomataPila` en `recognizer/automata.py` **hereda** de esa clase y agrega alfabeto de pila Γ, símbolo de fondo Z0 y transiciones `(estado, lectura|ε, cima) → {(destino, secuencia_a_apilar)}`. La pila es una tupla con la cima en el índice 0; reemplazar la cima por `()` desapila. EPSILON se toma del proyecto 1.

3.2 Definición formal

M = (Q, Σ, Γ, δ, q0, Z0, F) con Q = {q0, q1, q2}, Γ = V_G ∪ Σ ∪ {Z0}, F = {q2} y:

1. δ(q0, ε, Z0) = {(q1, S Z0)} — inicialización.
2. Para cada A → α en R: (q1, α) ∈ δ(q1, ε, A) — expansión de variables.
3. Para cada a ∈ Σ: δ(q1, a, a) = {(q1, ε)} — coincidencia de terminales.
4. δ(q1, ε, Z0) = {(q2, Z0)} — finalización.

Acepta por estado final con toda la entrada consumida. Invariante: en q1, si se leyó el prefijo u y la pila contiene β Z0, entonces S ⇒* uβ por una derivación izquierda; al aceptar, β = ε y S ⇒* w. Por tanto L(M) ⊆ L(G). Recíprocamente, cada derivación izquierda de una palabra de G se simula mediante las expansiones de su variable izquierda y la lectura de los terminales resultantes, hasta quedar Z0 y habilitar q2. Así L(G) ⊆ L(M) y L(M) = L(G). Es la construcción estándar CFG a PDA de HMU y Sipser; las notas de Stony Brook presentan esta misma estrategia de derivación izquierda.

Formalmente δ: Q × (Σ ∪ {ε}) × Γ → $\wp$(Q × Γ*); los casos no enumerados tienen conjunto de destinos vacío. La pila inicial es Z0 y su cima se escribe a la izquierda. En q2 queda el fondo: se acepta por estado final, no por vaciar la pila.

3.3 Simulación y evidencia

El algoritmo mantiene una cola con la configuración inicial (q0, 0, (Z0,)). Antes de expandir el frente comprueba aceptación y después el límite opcional. Para cada configuración extraída consulta las transiciones ε y, si queda entrada, las del siguiente token. Cada destino crea `apilar + pila[1:]`; la posición avanza solo con lectura. Primero aplica la poda, después descarta visitadas y finalmente guarda el padre, la transición y el sucesor en la cola. Si la cola se vacía devuelve rechazo. El límite cuenta expansiones y produce **inconclusa**. La poda se activa automáticamente solo en autómatas certificados por la construcción sin producciones vacías; en un PDA externo no se presume válida y no se garantiza terminación.

`recognizer/pda.py` ejecuta cualquier `AutomataPila` con búsqueda en anchura (BFS) sobre configuraciones (estado, posición, pila completa) y un conjunto de visitadas. Como G no tiene reglas vacías, cada símbolo sobre el fondo debe consumir al menos una palabra; una configuración con más símbolos pendientes que palabras restantes se poda con seguridad. Esto garantiza terminación a pesar de la recursión izquierda VP → VP PP. La evidencia contiene todas las configuraciones exploradas con su padre y transición, el recorrido aceptante validado o la causa del agotamiento.

Paso	Descripción instantánea	Acción
0	(q0, he cooks, Z0)	Inicio
1	(q1, he cooks, S Z0)	Inicializar
2	(q1, he cooks, NP VP Z0)	S → NP VP
3	(q1, he cooks, he VP Z0)	NP → he
4	(q1, cooks, VP Z0)	Leer he
5	(q1, cooks, cooks Z0)	VP → cooks
6	(q1, ε, Z0)	Leer cooks
7	(q2, ε, Z0)	Finalizar

Cada fila se relaciona con la siguiente por ⊢. Esta es la ruta aceptante reconstruida, no toda la exploración BFS: `he cooks` genera 15 configuraciones, expande 11, evalúa 14 transiciones, realiza 21 consultas, copia 50 símbolos y poda 3 candidatas; no encuentra repetidas.

4. Algoritmo CYK

T[i, l] es el conjunto de variables que derivan las l palabras desde la posición i. La base usa las reglas léxicas; para l = 2..n, cada corte c combina T[i, c] y T[i+c, l−c] con las reglas A → B C. Se acepta si S ∈ T[0, n]. Cada variable guarda su primer testigo (producción, corte), con lo cual se reconstruye un árbol y la derivación izquierda. Ejemplo `she drinks a beer`:

```
l=4 | S
l=3 | Ø       VP
l=2 | S       Ø       NP
l=1 | NP      V,VP    Det     N
      she     drinks  a       beer
```

S ⇒ NP VP ⇒ she VP ⇒ she V NP ⇒ she drinks NP ⇒ she drinks Det N ⇒ she drinks a N ⇒ she drinks a beer. En un rechazo (`he eats cake`) la tabla completa muestra que S no está en la celda final.

5. Ejecución

Python ≥ 3.10 para los reconocedores, con biblioteca estándar; verificado con Python 3.14. No se requiere instalar paquetes para reconocer ni ejecutar las pruebas. Desde la raíz del repositorio:

```
python main.py "she drinks a beer" --traza                # PDA y CYK, con pasos
python main.py "he eats cake" --metodo cyk --traza        # solo CYK
python main.py "he cooks" --json salidas/resultado.json   # evidencia completa en JSON
python -m unittest discover -s tests -v                   # todas las pruebas
python scripts/conteos_pda.py                             # ocho casos del PDA
python scripts/benchmark.py                               # regenera docs/operaciones.csv
python scripts/graficas.py                                # genera SVG sin dependencias
```

Códigos de salida: 0 aceptada, 1 rechazada, 2 inconclusa, error o discrepancia entre métodos.

Para generar el documento se necesita Chrome y la dependencia separada de `requirements-docs.txt`. En PowerShell, el entorno de documentación se crea fuera del repositorio:

```
python -m venv ../_coord-proyecto2/venv-docs
../_coord-proyecto2/venv-docs/Scripts/python.exe -m pip install -r requirements-docs.txt
../_coord-proyecto2/venv-docs/Scripts/python.exe scripts/generar_pdf.py
```

El comando habitual, con ese entorno activo, es `python scripts/generar_pdf.py`. Se producen `docs/informe.html` autónomo y `docs/informe.pdf`. El script busca Chrome en su ruta estándar de Windows; permite `--chrome RUTA` o la variable `CHROME_PATH`. No se requiere conexión a Internet al imprimir: las figuras se embeben en el HTML.

6. Tasas de crecimiento

6.1 CYK: desglose y cota ajustada

Con R_bin = 5 y R_lex = 25 reglas, los bucles recorren todas las celdas, cortes y reglas sin salidas tempranas:

Operación	**Conteo exacto**
Celdas	$n(n+1)/2$
Consultas léxicas	$25n$
Cortes	$\Sigma_{l=2..n} (n-l+1)(l-1) = (n^3 - n)/6$
Comprobaciones de reglas binarias	$5(n^3 - n)/6$

Con $j = l-1$, el conteo de cortes se obtiene de:

$\sum_{j=0}^{n-1} (n-j)j = n \cdot n(n-1)/2 - (n-1)n(2n-1)/6 = (n^3-n)/6.$

La suma seleccionada $C(n) = 5(n^3-n)/6 + 25n + n(n+1)/2$ pertenece a $\mathbf{\Theta(n^3)}$. No es el total de instrucciones ni la suma de todos los contadores: excluye, entre otras, las altas de variables y la reconstrucción. Los tres bucles de longitud, inicio y corte, más el recorrido completo de reglas binarias, justifican una cota ajustada para G. Las consultas a diccionarios se consideran O(1) esperado. El `and` de Python puede omitir la segunda consulta cuando falla la primera; las comprobaciones contadas sí son exactas.

Para gramática variable, el llenado cuesta $\Theta(n^2 + n\cdot|R_{lex}| + n^3\cdot|R_{bin}|)$, con $n \geq 2$. La forma convencional $\mathbf{O(n^3\cdot|G|)}$ es una cota superior; es $\mathbf{\Theta(n^3\cdot|G|)}$ cuando $|R_{bin}| = \Theta(|G|)$, como en G fija. No es una igualdad ajustada uniforme para una gramática dominada por reglas léxicas. La validación y ordenación inicial de reglas añade $O(|G| \log |G|)$ como cota superior. Para G esos costos son constantes.

La tabla ocupa $\Theta(n^2)$ celdas con hasta 8 variables y testigos cada una. El árbol ocupa $\Theta(n)$; la lista explícita de formas de la derivación y su reconstrucción requieren $O(n^2)$. Esto sigue dominado por $\Theta(n^3)$ en entradas largas de G. Para $n = 0$ todos los contadores son cero y se rechaza sin tabla.

6.2 PDA: cota exponencial en general, polinomial en G

Operación dominante elegida por David: símbolos de pila copiados al construir cada sucesor. Para una CFG fija sin ε con poda, el número de configuraciones está acotado por O(n·g^(n+1)), con g = |Γ|, de donde se obtiene una cota superior exponencial $O(n^2 \cdot g^{(n+1)})$. Para **G** toda pila alcanzable tiene la forma β PP^k Z0, con β en un conjunto finito y 0 ≤ k ≤ n; hay O(n) pilas y $O(n^2)$ configuraciones. Cotas superiores para G: **$O(n^2)$ transiciones evaluadas** y **$O(n^3)$ símbolos copiados** (y memoria $O(n^3)$, porque se guardan las pilas completas como evidencia). Las cotas anteriores son **superiores**, no tasas Θ demostradas. Suponen gramática fija, ramificación constante y consultas hash amortizadas. En general, $C \le |Q|(n+1)\sum_{h=0}^{n+1} g^h$ configuraciones; cada una genera O(1) sucesores y cada pila cuesta O(n) al copiarse. Para G, la forma β PP^k Z0 reduce a O(n) las pilas posibles. β pertenece a un conjunto finito: VP, V NP, verbo NP, verbo, NP, Det N, determinante N, N, sustantivo, P NP, preposición NP, pronombre o ε; las etapas del sujeto también forman un conjunto constante. Multiplicar por las O(n) posiciones da $O(n^2)$ configuraciones y por O(n) de copia da $O(n^3)$.

En CNF, n reglas léxicas y n−1 binarias producen 2n−1 expansiones; al sumar n lecturas y dos pasos administrativos, cualquier ruta aceptante tiene **3n+1 transiciones**. Esto no hace lineal a `validar_recorrido`: compara las pilas explícitas antes y después, por lo que su costo es O(suma de longitudes de pila), a lo sumo $O(n^2)$, además de recibir el recorrido. Buscar la ruta con BFS tiene los costos superiores ya derivados.

Contador PDA	Qué mide
`configuraciones_generadas`	Inicial más cada candidata, aun podada o repetida
`configuraciones_expandidas`	Configuraciones retiradas para buscar sucesores
`consultas_transiciones`	Búsquedas en δ, para ε y siguiente token disponible
`transiciones_evaluadas`	Destinos examinados de esas consultas
`simbolos_copiados_pila`	1 inicial + suma de (longitud anterior−1 + longitud nueva)
`podas`, `repetidas`	Candidatas descartadas por longitud o ya visitadas

Las copias cuentan referencias de la cola de la tupla y su concatenación. No incluyen todas las copias del registro ni el costo exacto de hash: miden el crecimiento de la representación. Se cumple generadas = 1 + transiciones evaluadas. CYK cuenta además `reglas_satisfechas` (coincidencias léxicas/binarias) y `variables_agregadas` (primer testigo en cada celda).

7. Experimentos reproducibles

`scripts/benchmark.py` ejecuta ambos métodos sin límite de búsqueda y escribe `docs/operaciones.csv` (40 filas, determinista; no mide tiempos). Familias, con F_k = `she eats the cake` + k veces `with a fork` (n = 3k + 4):

Familia	Frase	k	Decisión
F	F_k	0..15	aceptada
R	F_k + `with` (PP incompleto)	0..15	rechazada
E	`cooks` + F_k (orden inválido desde el inicio)	0, 5, 10, 15	rechazada
D	F_k + `dances` (palabra fuera de Σ)	0, 5, 10, 15	rechazada

Columnas: `pda_*` son los contadores de `docs/pda.md`; `cyk_*` los de `docs/cyk.md`; `pda_causa` indica `frontera_agotada` o `fuera_del_alfabeto`; `pda_limite` vacío significa que no hubo límite y, por tanto, ningún resultado es inconcluso. `coinciden` es `True` en las 40 filas.

n	PDA transiciones	PDA copias de pila	CYK comprobaciones	CYK variables agregadas
4 (F_0)	66	304	50	9
7 (F_1)	196	1 323	280	16
19 (F_5)	1 376	20 729	5 700	44
34 (F_10)	4 336	109 179	32 725	79
49 (F_15)	8 946	315 154	98 000	114
50 (R_15)	9 310	334 274	104 125	115
50 (E_15)	7	37	104 125	117
50 (D_15)	0	0	104 125	114

En las 16 filas de F, los conteos coinciden exactamente con estos polinomios (y análogos en R):

- PDA, transiciones evaluadas: $(11/3)n^2 + 3n - 14/3$.
- PDA, símbolos copiados: $(22/9)n^3 + (71/6)n^2 - (107/6)n + 266/9$.
- CYK, comprobaciones binarias: $(5/6)(n^3 - n)$, igual que la fórmula analítica.

7.1 Gráficas del conteo dominante

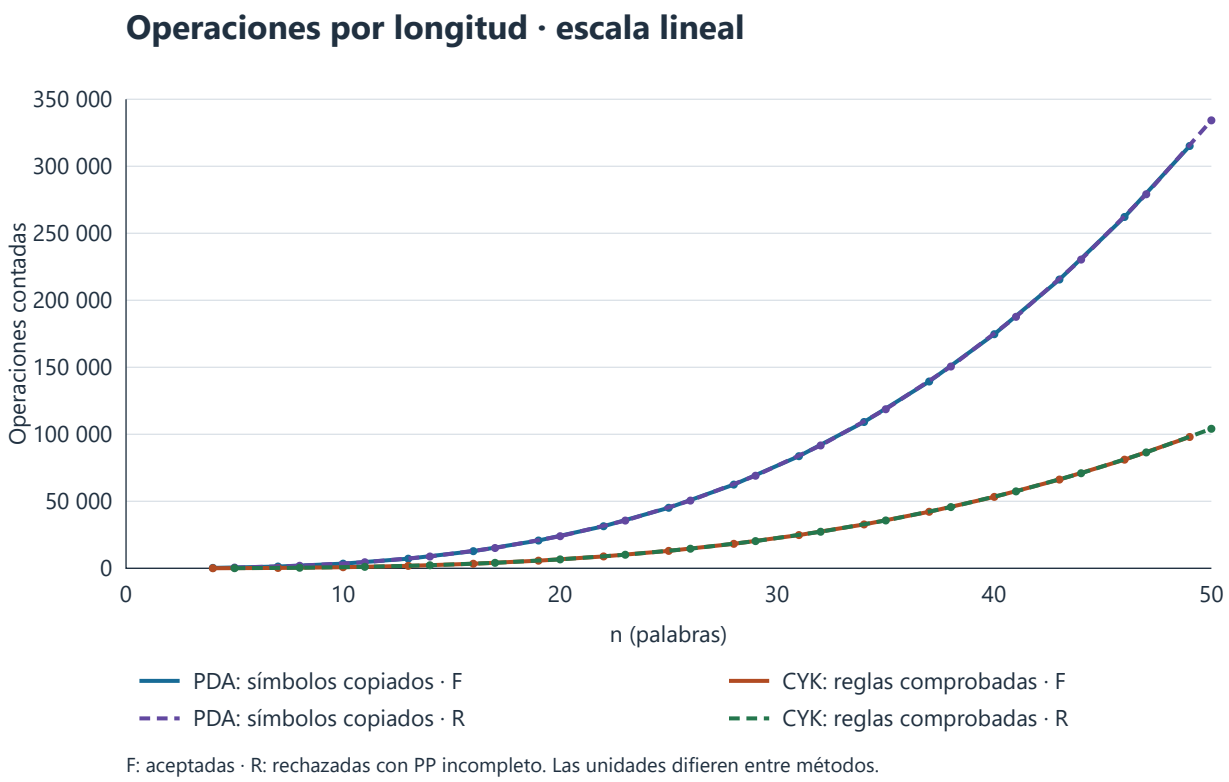


Figura 1. Conteos del CSV para F (aceptadas) y R (rechazadas). Se usan las copias de símbolos del PDA y las comprobaciones binarias de CYK. Las operaciones de distintos programas no cuestan lo mismo; la altura de las curvas no equivale a una comparación de segundos.

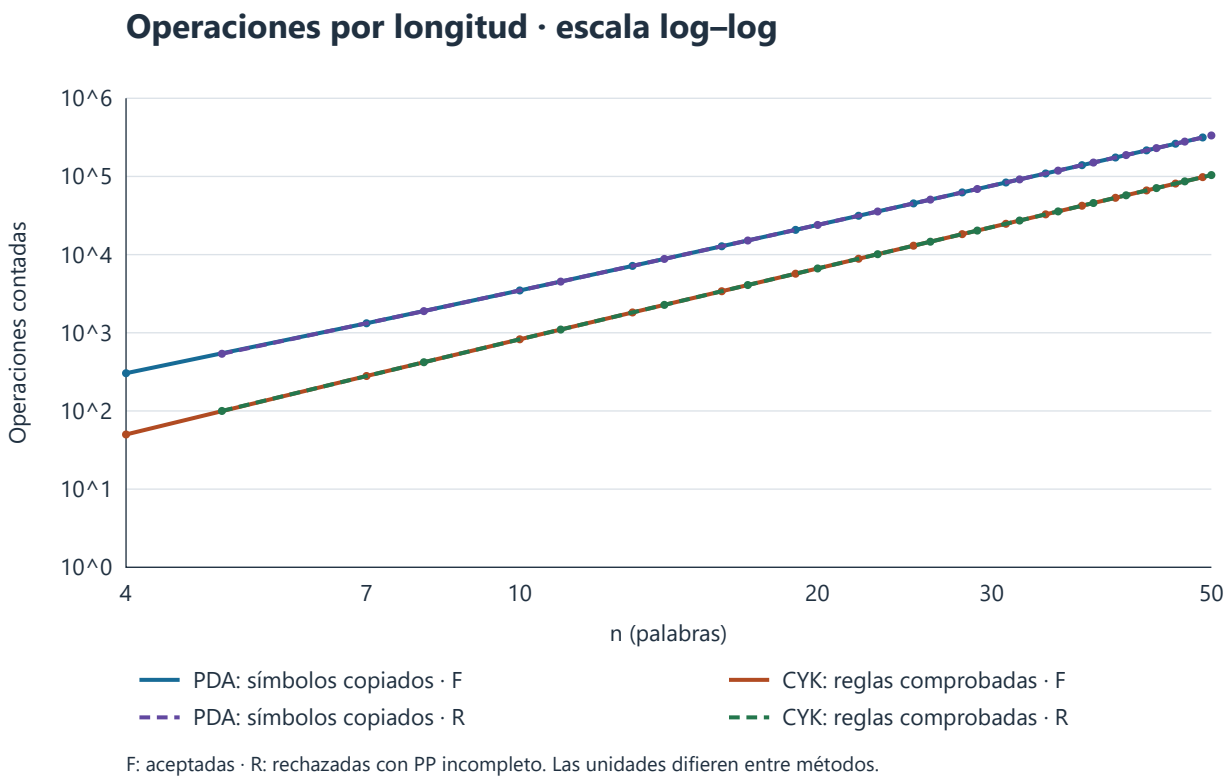


Figura 2. Los mismos datos con ambos ejes logarítmicos, incluidos los casos cortos. Las pendientes se aproximan a 3 en entradas largas de estas familias, coherentes con términos cúbicos; la evidencia finita no demuestra una cota ajustada del PDA.

8. Discusión de desempeños

Lo que dice el análisis. CYK es $\Theta(n^3)$ para cualquier gramática CNF fija y cualquier entrada: su costo es el de llenar la tabla, no el de la frase. El PDA no determinista simulado por búsqueda no tiene, en general, una cota polinomial: para una CFG arbitraria la cota superior es exponencial, y sin visitados ni poda la recursión izquierda lo haría no terminar. En G, la forma restringida de las pilas reduce el espacio de configuraciones a $O(n^2)$; por eso el PDA resulta polinomial aquí. Esa es una propiedad de G, no del método.

Lo que muestran los datos. Medido en «decisiones» (transiciones del PDA frente a comprobaciones de reglas de CYK), el PDA crece como n^2 y CYK como n^3: el PDA hace más trabajo con n = 4 (66 frente a 50) y menos desde n = 7 (196 frente a 280); con n = 49, 8 946 frente a 98 000. Medido en el trabajo real de la representación del PDA, cada transición copia y compara una pila de longitud O(n), y los símbolos copiados crecen como $(22/9)n^3$, unas 2,9 veces las comprobaciones de CYK para n grande. **Las unidades no son equivalentes**: una comprobación de CYK hace hasta dos consultas a diccionarios; una transición del PDA construye, hashea y registra una tupla. Ninguna de las dos métricas por sí sola decide cuál programa es «más rápido».

Aceptación y rechazo. CYK conserva los conteos de celdas, cortes y comprobaciones para frases de igual n (filas R, E y D con n = 50: 104 125 comprobaciones; F llega a n = 49). El PDA depende de la entrada: rechaza `cooks she eats…` tras 7 transiciones porque ninguna expansión de S puede empezar con `cooks`, y rechaza palabras fuera de Σ antes de explorar (0 operaciones). En cambio, un rechazo tardío (familia R) cuesta prácticamente lo mismo que una aceptación de igual longitud: el polinomio de F evaluado en n = 50 da 9 312 transiciones y R_15 usa 9 310. BFS agota la frontera al rechazar, y al aceptar llega a la configuración final cuando ya exploró casi todo el espacio alcanzable.

Recursión izquierda. VP → VP PP obliga al PDA descendente a usar visitados y poda por longitud para terminar; CYK, ascendente, no se ve afectado porque cada celda combina segmentos más cortos.

Reutilización de subproblemas. CYK calcula cada segmento una sola vez (programación dinámica). El PDA no reutiliza el análisis de un segmento: el mismo NP puede analizarse de nuevo con distintos sufijos PP^k en la pila, porque las configuraciones difieren. En las 40 filas se observa `repetidas = 0`: no hay configuraciones idénticas que aprovechar. A la vez, la tabla de CYK es muy dispersa en G ($(7n-1)/3$ variables agregadas en F, frente a $n(n+1)/2$ celdas): CYK paga por subproblemas que casi nunca producen algo.

Memoria. CYK: $\Theta(n^2)$ celdas. PDA: $O(n^2)$ configuraciones con pilas de longitud O(n), $O(n^3)$ símbolos guardados como evidencia en G, y exponencial en el peor caso general.

Límites de esta evidencia. Los experimentos usan familias concretas con $n \leq 50$. Que los conteos coincidan con polinomios en 16 puntos respalda las cotas, pero no las demuestra: la demostración es el análisis de las secciones 6.1 y 6.2. Los conteos no miden constantes de Python ni de la máquina, y no se usaron tiempos de reloj. No puede concluirse que un método sea siempre mejor: para G y frases largas el PDA hace menos transiciones, pero CYK ofrece una garantía $\Theta(n^3)$ para cualquier gramática CNF, que la búsqueda en el PDA no tiene.

9. Validación

`python -m unittest discover -s tests`: **57 pruebas OK**, verificadas en esta entrega; pruebas de gramática, modelo de autómata, PDA, CYK, consola e integración. CYK se comprueba con tablas calculadas a mano (`he cooks`, `she drinks a beer`), conteos cerrados, árboles cuyas hojas son exactamente los tokens y cuyas producciones pertenecen a G, derivaciones izquierdas válidas y una gramática CNF distinta (a^nb^n). La validación cruzada compara las decisiones del PDA y CYK en todas las cadenas de longitud ≤ 5 sobre {he, cooks, eats, with, the, cat} (9 331 cadenas) y en 60 frases generadas de G con semilla fija junto con versiones perturbadas. Es evidencia acotada a esas longitudes y alfabeto.

Casos mínimos comprobados desde la consola y las pruebas:

Entrada	Resultado esperado de ambos
`he cooks`	Aceptada; 7 transiciones en la ruta PDA
`she drinks a beer`	Aceptada; S en T[0,4]
`he eats cake`	Rechazada; falta determinante
`he cooks with`	Rechazada; PP incompleto
Cadena vacía o `he`	Rechazada
`he dances`, `He cooks`, `he cooks.`	Rechazada; token fuera de Σ
`he cooks`	Aceptada tras separar espacios

Con `python main.py "he cooks" --metodo pda --limite 1` se obtiene inconclusa y salida 2. El límite no se aplica a CYK ni convierte una búsqueda interrumpida en rechazo. El benchmark contiene 16 aceptaciones y 24 rechazos, sin límites ni discrepancias. Regenerarlo conserva los datos originales; solo cambia CRLF a LF.

10. Referencias

- Hopcroft, J., Motwani, R. y Ullman, J. *Introduction to Automata Theory, Languages, and Computation*, 3.ª ed. Temas: equivalencia CFG–PDA y prueba de pertenencia mediante CYK.
- Sipser, M. *Introduction to the Theory of Computation*, 3.ª ed. Temas: autómatas de pila, gramáticas libres de contexto y programación dinámica.
- CYK algorithm, enlazado en el enunciado.
- Documentación del proyecto: `docs/acuerdos.md`, `docs/pda.md`, `docs/cyk.md`, `docs/operaciones.csv`.
- Stony Brook University. Equivalence of Pushdown Automata with Context-Free Grammar. Material universitario consultado para la construcción CFG–PDA.
- Universidad del Valle de Guatemala. *Proyecto 2 2026*, enunciado y rúbrica del curso CC2019, archivo `Proyecto 2 2026.pdf` del repositorio.